AI-Native SaaS Development — Strategic & Technical Outline

Framework for building AI-powered MVPs and production-ready SaaS (15–20 day build cycles), including an MCP (Model Context Protocol) layer for tool/data connectivity.

1. Core Philosophy

Principle	What It Means
Deterministic core + probabilistic edge	Business-critical logic (billing, eligibility, tax, legal thresholds) stays as deterministic code. The LLM handles judgment/extraction/reasoning. Model recommends; rules layer confirms — especially for regulated decisions.
Workflows vs. agents	Most problems need an orchestrated pipeline (extract → reason → verify → act), not a fully autonomous agent. Reserve agentic loops for genuinely unpredictable branching.
Evaluation-driven development	AI software needs eval sets, golden datasets, and regression testing before shipping — otherwise a prompt change can silently break behavior.
Explainability by design	High-stakes decisions must produce a stated reason, not just an answer — a trust and often compliance requirement.
Scope discipline	15–20 day builds require one user journey, one model provider, one deployment target, synthetic data validated before real data.

2. Two Reference Use Cases

A. Insurance claim eligibility from accident images — multimodal reasoning over photos + RAG against the policy document + structured output with a mandatory reasoning field. Deterministic layer: policy terms/thresholds/exclusions as rules. Fine-tuning not needed initially.

B. Short-term rental listing tracker (tax compliance) — mainly a data-pipeline/change-detection problem, not an LLM problem. AI role is narrow: parsing listing text, flagging relisting patterns. Caution: scraping Airbnb/Booking.com raises real legal exposure (ToS, CFAA-adjacent risk) — prefer licensed data providers (AirDNA, Rabbu) or official APIs.

> **Takeaway:** identify which part of the problem is a genuine reasoning/judgment task (LLM's job) vs. a data-engineering or legal/compliance task (traditional engineering's job).

3. Technology Stack

3.1 Frontend

Next.js (React) with the Vercel AI SDK for streaming; Tailwind + shadcn/ui; TanStack Query for server state. Streamlit/Retool only for internal pre-MVP prototyping, not production.

3.2 Backend & Orchestration

FastAPI (Python) if AI logic is also Python, or NestJS/Express if Node-first. Background jobs (Celery+Redis or BullMQ) are non-negotiable — image/document processing must not block requests. Orchestration: LangGraph or Anthropic's native tool-use/Agent SDK for multi-step logic; LlamaIndex for RAG. Keep this layer thin and swappable.

MCP (Model Context Protocol) — Anthropic's open protocol standardizing how an LLM app connects to external tools, data, and prompt templates, replacing one-off integrations (turns "M × N" into "M + N"). An MCP client lives inside the app (Claude Desktop, Claude Code, custom runtime); an MCP server exposes tools (model-controlled), resources (app-controlled data), and prompts (user-controlled templates) — locally as a subprocess or remotely. Fits both use cases above: expose policy/claims lookups or licensed rental-data APIs as MCP tools/resources instead of custom glue. Claude Code is itself an MCP client, so connecting it to a GitHub/Drive/Figma/DB server gives coding sessions live project context. Roadmap: multi-agent architecture, registry/discovery, stronger auth — treat as an evolving ecosystem and pin versions. [NEW]

3.3 Data, Hosting & Security

- **Databases:** PostgreSQL (Supabase/Neon) as system of record; pgvector to start, Pinecone/Weaviate/Qdrant at scale; S3/R2 for objects; Redis for cache/queues.
- **Hosting:** Vercel for frontend/edge; Railway/Render/AWS ECS-Fargate for heavier backend/AI workloads. Anthropic API as primary model access, abstracted behind an interface.
- **Observability:** LangSmith/Helicone/Braintrust for LLM tracing; Sentry/Datadog/Grafana for standard APM. GitHub Actions for CI/CD.
- **Security:** encryption at rest, strict PII handling, audit logs for every AI decision, human-in-the-loop review for high-stakes output. **Treat each connected MCP server as a trust boundary — scope permissions narrowly, review third-party servers, log every tool call.** [NEW]

4. Claude Code Workflow

- Spec before code — a short markdown spec (flow, data model, API contracts) beats open-ended prompting.
- Build in vertical slices (UI → API → DB → AI call) for a demoable product daily, not horizontal layers.
- Highest value: boilerplate, scaffolding, tests, refactors, debugging, eval scripts, migrations.
- Maintain a CLAUDE.md at repo root for conventions — the top consistency practice across sessions.
- Git discipline: small commits, feature branches, easy rollback.
- Review diffs like a capable engineer's work, don't blind-accept — especially business logic/security paths.
- **Connect MCP servers (GitHub, Drive, Figma, internal DB) so scaffolding is grounded in real repo/data context.** [NEW]

5. LLM Strategy Ladder

Lvl	Approach	When to Use	Core Skill
1	Prompt engineering	Starting point for nearly every MVP	Cheap, highest early ROI
2	RAG — ground in your own data	Answers must cite source documents	Vector DBs, chunking, retrieval eval
3	Tool use / agents (typically via MCP servers)	Both reference use cases require this	Schema design, orchestration, failure handling
4	Fine-tuning	Only once prompting+RAG plateau, thousands of labeled examples	ML fundamentals, data curation, evaluation
5	Train a foundation model	Not realistic for a product company	N/A

Most enterprise SaaS value is created at Levels 1–3; fine-tuning is a deliberate, resource-heavy decision, not a default.

6. Tooling Checklist

- Claude Code — implementation
- Claude Console/API — prompt iteration & eval
- **MCP servers (filesystem, fetch, GitHub, Drive, Figma, or custom)** [NEW]
- **MCP Inspector — testing/debugging servers** [NEW]
- LangGraph / native tool-use — orchestration
- LlamaIndex or custom pipeline — RAG

- PostgreSQL + pgvector, Redis
- Playwright/Puppeteer (only where scraping is legal)
- LangSmith/Braintrust — eval & observability
- Docker — consistent environments
- Vercel/Railway/AWS — deployment
- Figma — UI iteration ahead of implementation

7. Where Outside Expertise Is Needed

- **Eval design / AI quality measurement** — the most valuable specialist skill to bring in early.
- **Legal/compliance review** — insurance (regulated, explainability) and rental tracker (ToS/scraping exposure).
- **ML support for fine-tuning** — only once you reach Level 4; failure modes aren't obvious from SWE experience alone.
- **AI security review** — prompt injection is a growing attack surface, **especially with third-party MCP servers granted tool-calling permissions** **[NEW]**.
- **Infra/cost architecture** — LLM API costs scale unpredictably; bring in a fractional advisor once several customers are live.

8. Learning Path (Suggested Sequence)

1. **Prompting fundamentals** (~1 wk, free) — Anthropic's Prompt Engineering Tutorial (docs.anthropic.com); Anthropic's GitHub courses repo; DeepLearning.AI's "ChatGPT Prompt Engineering for Developers."
2. **Claude API + Claude Code** (~1–2 wks) — DeepLearning.AI × Anthropic "Claude Code: A Highly Agentic Coding Assistant" (Elie Schoppik) — CLAUDE.md, subagents, worktrees, MCP integration; DeepLearning.AI "Agent Skills with Anthropic"; Anthropic Academy (anthropic.com/learn).
3. **RAG** (~1–2 wks) — DeepLearning.AI "Building and Evaluating Advanced RAG" / "Retrieval Augmented Generation" (Zain Hasan); freeCodeCamp "RAG From Scratch"; LlamaIndex docs.
4. **MCP (~3–5 days) — DeepLearning.AI × Anthropic "MCP: Build Rich-Context AI Apps with Anthropic" (Elie Schoppik, free, ~1.5 hrs — filesystem/fetch servers, Claude Desktop as client, remote deploy, MCP Inspector); Anthropic Academy "Introduction to Model Context Protocol" (build servers/clients in Python); official spec at modelcontextprotocol.io; reference-servers repo on GitHub.** **[NEW]**
5. **Agents/orchestration** (~2–3 wks) — DeepLearning.AI "AI Agents in LangGraph"; LangChain YouTube channel; Cole Medin (YouTube) for practical agent builds; Anthropic's "Tool Use" course. Ideally call tools through the MCP server built in step 4.
6. **Evaluation & observability** — introduced alongside steps 3–5, not after. DeepLearning.AI "LLMOps" / "Automated Testing for LLMOps"; LangSmith and Braintrust docs.
7. **Fine-tuning** — only if a use case demonstrably plateaus on prompting+RAG. DeepLearning.AI fine-tuning/quantization courses; pair with *AI Engineering* (Chip Huyen) and *The LLM Engineering Handbook* (Iusztin & Labonne).